School of Computing

FACULTY OF ENGINEERING AND
PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Final Report

An investigation into the effectiveness of various algorithms to help check spelling of English words.

Matthew Alain Camus

Submitted in accordance with the requirements for the degree of Meng/BSc Computer Science

2023/24

COMP3931

The candidate confirms that the following have been submitted*:*

Items	**Format**	**Recipient(s) and Date**
Final Report	*PDF file*	*Uploaded to Minerva (01/05/24)*
Link to online code repository	*URL*	*Sent to supervisor and assessor (01/05/24)*

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of student) 

Summary

In this report, an attempt was made to establish an understanding of how spell checkers work at a fundamental level and analyse how they find words that are misspelled. For this, research was done into various search methods and algorithms of measuring string similarity. To help analyse the use of these algorithms, a system was implemented which collects words from a corpus and stores them. Then a spell checker program would use these to see if a given word is valid or not. This task naturally allowed for the further analysis of other searching and storing algorithms for collecting and then finding words in efficient ways.

To fulfil this goal, two separate systems were created to test the capabilities of the various algorithms and search methods available. One was written entirely in C, the other in Python. Both kept the exact same design and were tested with the same data to gauge the viability of each for the role of an industry spell checker. Once development finished, a large set of misspelled words were used to test the applications. The generated results were then processed by a separately developed graphing program to visualise the runtimes and corrections that they gave to typos.

After analysing the results of the automated testing, it was immediately clear that the best system was the one written in C. This produced considerably faster runtimes and was therefore more suitable for the job of a spell checker. The fastest search method was found to be the binary search method, which could make use of a sorted dictionary to get results quickly. The most optimal algorithm was harder to ascertain, given that many performed better with some metrics and worse with others. However, the Jaro-Winkler algorithm ended up being selected as the best general-purpose algorithm for its short runtimes and reliable suggestions to misspelled words.

Acknowledgements

This project could not have been possible without the help of supervisor Ammar Alsalka, who gave invaluable pointers and encouragement throughout the development lifecycle. It should also be noted that the advice received from assessor Brandon Bennett inspired the inclusion of the Python version of the software. This was incredibly useful in expanding and providing new ideas for the software.

A thank you should be given to the British National Corpus, who's texts were useful in building dictionaries for the spell checker to use when comparing words. Without this the system couldn't function. A special thanks should also be given to Roger Mitton whose collection of misspelled words where vital in providing exhaustive test data so that the capabilities of each algorithm could be gauged. Finally, a mention should be given to dwyl on GitHub, who provided the precompiled list of English words that were used by the prototype.

Table of Contents

Chapter 1
Introduction and Background Research

1.1 Introduction

The use of spell checking in modern day computer systems is almost universal. From search engines to coding software, people have become reliant on computers to be able to not only fix spelling mistakes as they go along with their work, but even offer suggestions to better grammar. The benefits of such mechanisms are that they reduce the workload for the person writing on their machine. Not only can it notify them of typing misspelt words, but in some cases predict what they are meaning to say and therefore shorten the time to complete their work.

Such widespread use of this kind of system requires a large amount of word processing, which needs to be done in real time. As an example, the Oxford English Dictionary [1] says that there are over 500,000 words in the English language that are in currently in use. As such, a spell-checking system must have the capacity to store all these words and provide checks in a short window of time to maximise the efficiency of a computer user.

Therefore, this project is dedicated to using various ways of providing spell checks for English words. To accomplish this, two applications have been made which can process a set of English texts, recording the words within them. It can then use those words to analyse the speed and accuracy of a spell check, to assess which methods produce the best results.

1.2 Overview of algorithms

As the primary means of testing how similar two strings are, a series of string metric or string similarity metric algorithms were chosen. These algorithms are designed to measure the distance between two strings. This is useful in our case as the program can take a given string, and then check its similarity to all other strings to give optimal suggestions to correct misspellings. Some of these same algorithms are heavily used in not only natural language processing systems, but also in other fields such as image analysis, machine learning, data mining and many other integrations.

For measuring the time and space complexity of the following algorithms, the letters ‘n’ and ‘m’ will be used to refer to the lengths of both the first and second strings respectively.

1.2.1 Levenshtein Distance

Levenshtein distance is an algorithm used to measure the distance / difference between two strings, represented as a single integer value. The algorithm used is iterative and works as described in figure 1, where a full 2-D matrix stores the results of calculations. It then goes through both strings comparing characters and updates the matrix based on the smallest of 3 actions from: inserting, deleting, or substituting a character.

```
Algorithm LevenshteinDistance is
        input: strings: A[0 ... length(a)], B[0 ... length(b)]
        output: integer: distance

        let CalculationMatrix be a 2-d array of zeros, dimensions length(A)+1, length(B)+1

        //initialise the first values of the matrix
        for the length of string A do
                set the first value of each row of the CalculationMatrix to be the value of its index
        for the length of string B do
                set the first value of each column of the CalculationMatrix to be the value of its index

        for the length of string A do
                for the length of string B do
                        if current character of string A = current character of stirng B then
                                set current CalculationMatrix value to the previous CalculationMatrix value
                        else
                                set current CalculationMatrix[i][j] value to 1 + minimum cost process between insertion, deletion or substitution

        return last value in CalculationMatrix
```

Figure 1. Levenstein Distance Algorithm pseudocode

In other words, the algorithm assesses how many steps it would take to transform the first string to the second string, and the number of steps is the distance between them. The larger the distance, the greater the difference between the two strings.

The time complexity of this algorithm is O(n * m). This is because there is a nested loop where n * m iterations are needed to compute a complete result. The space complexity is also O(n * m) since a full matrix of dimensions n * m is needed to store the result of all calculations.

1.2.2 Optimal String Alignment (OSA) Distance

```
Algorithm OptimalStringAlignmentDistance is
        input: strings A[0 ... length(A)], B[0 ... length(B)]
        output: integer: distance

        let CalculationMatrix be a 2D array of integers, dimensions legnth(A) + 1, length(B) + 1

        //initialise the first values of the matrix
        for the length of string A do
                set the first value of each row of the CalculationMatrix to be the value of its index
        for the length of string B do
                set the first value of each column of the CalculationMatrix to be the value of its index

        for the length of string A do
                for the length of string B do
                        if current character of string A = current character of stirng B then
                                let Cost := 0
                        else
                                let Cost := 1

                        set current CalculationMatrix[i][j] value to minimum cost process between insertion, deletion or substitution

                        if a transposition is possible to perform then
                                set current CalculationMatrix[i][j] value to minimum cost between current value or cost of a transposition

        return last value in CalculationMatrix
```

Figure 2. Optimal String Alignment Algorithm (OSA) pseudocode

This algorithm outlined in figure 2 is a step up from the Levenshtein algorithm, in that it allows transpositions to be included into the list of edit operations. This allows for the swapping of two adjacent characters in a string. However, it isn't allowed for multiple transforms to be applied to the same substring. Again, the larger the distance, the bigger the difference in the words.

This algorithm is also iterative and uses a full matrix for storing results of calculations, giving it a time complexity of O(n*m) and a space complexity of O(n*m). However, the extra tests for transpositions will mean the process will take slightly more time than the standard Levenshtein method.

1.2.3 Damerau Levenshtein Distance

```
Algorithm DamerauLevenshteinDistance is
        input: strings A[0 .. length(A)], B[0 ... length(B)]
        output: integer: distance

        let CalculationMatrix be a 2D array of integers, dimensions length(A) + 2, length(B) + 2

        //maximum length a transposition can occur
        let maxDistance = length(A) + length(B)

        for the length of string A do
                set first value of each row in CalculationMatrix to be maxDistance
                set second value of each row in CalculationMatrix to be the value of its index
        for the length of string B do
                set first value of each column in CalculationMatrix to be maxDistance
                set second value of each column in CalculationMatrix to be the value of its index


        for the length of string A + 1 do
                for the length of string B + 1 do

                        set current CalculationMatrix value to minimum cost process between insertion, deletion, substitution or transposition

        return last value in CalculationMatrix
```

Figure 3. Damerau Levenshtein Distance Algorithm pseudocode

This algorithm outlined in figure 3 is a further advancement on the OSA Algorithm, as it fully incorporates the use of transpositions, which can be done on the same substring. As before, the larger the final distance, the more dissimilar the two strings are.

This is still an iterative algorithm, with a time complexity of O(n * m) given its use of a nested loop to go through both strings. Similarly, the average space complexity is O(n * m) given that the data still needs to be stored in a full matrix.

1.2.4 Hamming Distance

```
Algorithm HammingDistance is
        input: strings A[0 ... length(A)], B[0 ... length(B)]
        output: distance: integer

        if length(A) doesn't = length(B) then
                return error

        let score := 0

        for the length(A) do
                if the next character in A doesn't = the equivalent character in B then
                        increment score by 1

        return score
```

Figure 4. Hamming Distance Algorithm pseudocode

The Hamming Distance algorithm outlined in figure 4 is the simplest algorithm tested, given that it only works on strings of the same length. This method goes through both words, updating a local count if any characters are different at a certain index. The final count is the distance between the two strings. The larger the score, the larger the string divergence. However, the fact that the two strings need to be the same length means the algorithm cannot correct errors in the case of where a letter is missing or needed.

This algorithm has a time complexity of O(n) from looping through one string linearly and a space complexity of O(1) to store the score variable, making it the most efficient algorithm in terms of both time and space.

1.2.5 Sørensen–Dice Coefficient

```
Algorithm Sørensen-Dice Coefficient is
        input: strings A[0 ... length(A)], B[0 ... length(B)]
        output: distance: float

        //score counter for how many bigram are matching
        let matches := 0

        let i := 0
        let j := 0

        //loop though both strings, comparing the bigrams for matches
        while i < length(A) - 1 and j < length(B) - 1 do

                let bigram1 = next two characters from string A
                let bigram2 = next two characters from string B

                if bigram1 = bigram2 then
                        increment matches by 1

                increment i by 1
                increment j by 1


        return (2 * matches) / ( (length(A) - 1) + (length(B) - 1 ) )
```

Figure 5. Sørensen–Dice Coefficient Algorithm pseudocode

The Sørensen–Dice algorithm outlined in figure 5 is like Hamming distance. However instead of comparing individual characters it compares their bigrams, or pairs of adjacent characters. It also differs by being able to handle strings of different lengths. Once the number of similar bigrams is counted, the overall score of similarity ‘s’ is calculated using the following formula:

$$s = \frac{2n_t}{n_a + n_b}$$

Where n_t is the is the number of bigrams found in both strings, n_a is the number of bigrams in string a, and n_b is the number of bigrams in string b.

This method means values are in the range of 0 to 1, where the closer the coefficient returned is to 1, the closer the two strings are to each other.

The time complexity would be O(m + n) to complete the calculation loop, with the space complexity is O(1) to store the variables for calculation.

1.2.4 Jaro Distance

```
Algorithm JaroDistance is
        input: strings A[0 ... length(A)], B[0 ... length(B)]
        output: distance: float

        //maximum distance up to which matching is allowed
        let maxDistance := (max(length(A), length(B)) / 2) - 1

        //count of matches
        let matches:= 0

        let string1Hash[0 ... length(a)] be an array of zeros, dimensions length(a)
        let string2Hash[0 ... length(b)] be an array of zeros, dimensions length(b)

        //traverse through string A
        for length(A) do

                //check if there are any matches
                for j := max( 0, i - maxDistance) till min(length(B), i + maxDistance + 1) do

                        if the next character in A = the next character in B and the next value in string2Hash = 0 then
                                let the next value in string1Hash := 1
                                let the next value in string2Hash := 1
                                increment matches by 1
                                break from for loop

        if matches = 0 then
                return 0.0

        //number of transpositions
        let t := 0
        let point := 0

        //count number of occurences where two characters match but there is a third matched character
        //in between the indices
        for length(A) inclusive do
                if the next value in string1Hash = 1 then

                        //find the next matched character in the second string
                        while string2Hash[point] = 0 do
                                increment point by 1

                        if the next value in A doesn't = b[point] then
                                increment point by 1
                                increment t by 1
                        otherwise then
                                increment point by 1

                t := t / 2

        return ((matches / length(a) + matches / length(b) + (matches - t) / matches) / 3.0)
```

Figure 6. Jaro Distance algorithm pseudocode

The Jaro distance algorithm, as outlined in figure 6, scores strings on a scale of 0 to 1. As before, the more similar the words are, the closer the final value is to 1. It works by counting the number of matching characters between the two strings, which are considered only if

they are the same and not further apart than $\left\lfloor \frac{\max(|s_1||s_2|}{2} \right\rfloor - 1$ characters apart, where s_1 and s_2 are strings 1 and strings 2 respectively. If there are some matching characters found using this process, then the number of transpositions is calculated. These are regarded as matching characters that are not in the right order. The result is then calculated as:

$$sim_j = \begin{cases} 0 & if\ m = 0 \\ \frac{1}{3}\left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m}\right) & otherwise \end{cases}$$

Where $|s_i|$ is the length of the string, m is the number of matching characters, and t is the number of transpositions.

This algorithm has a worst-case time complexity of O(n * m) with a space complexity of O(n + m).

1.2.4 Jaro-Winkler Distance

```
Algorithm JaroWinklerDistance is
        input: strings A[0 ... length(A)], B[0 ... length(B)]
        output: distance: float

        //get the standard Jaro distance
        let jaro_distance := jaroDistance(A, B)

        //if the jaro similarity is above a threshold
        if jaro_distance > 0.7 then

                //find the length of a common prefix
                let prefix := 0

                for i := 0 to min(length(a), length(b)) do

                        if the next character of A = the next character of B then
                                increment prefix by 1
                        else then
                                break from the loop


                prefix = min(4, prefix)

                jaro_distance = jaro_distance + 0.1 * prefix * (1 - jaro_distance)

        return jaro_distance
```

Figure 7. Jaro-Winkler Distance Algorithm pseudocode

The Jaro-Winkler Distance algorithm, as outlined in figure 7, is an advancement of the Jaro Distance method. It builds upon the values generated from the original by using a prefix scale, which gives strings more favourable scores that match from the beginning for a set prefix length.

So, the new result coefficient for two strings is:

$$sim_{jw} = sim_j + lp(1 - sim_j)$$

Where sim_j is the original Jaro distance, p is the prefix scale, and l is the prefix length.

However, this algorithm does not affect the overall time and space complexities significantly, which remain O(n * m) and O(n + m) respectively.

1.3 Overview of search methods

To look for certain words in the program, a search method must be used. These methods depend on the organisation of the words as to how they could be extracted and exploited for greater performance. This would therefore reduce wasted time on checking redundant strings.

In this section, ‘n’ refers to the number of words in the search method’s data structure.

1.3.1 Linear Search

The most basic approach for checking if a user’s word exists in the system is by using a linear search method.

In this system, words would be organised into a long 1-dimensional array. When a word needs to be checked, each item would be looked at from the first index, till either it is found, or the end of the array is reached.

This gives an average runtime of O(n).

1.3.1 Binary Search

Another approach to searching is with the binary search. This method works the same with linear search, in that it requires the words to be organised in a long 1-dimensional array. However, for the method to work it would require the array to be sorted in alphabetical order.

This method would be recursive and constantly check the middle of an increasingly small array for the correct string. This gives an average runtime of O(log n).

1.3.1 Hash Tables and Linked Lists

The final approach to searching is with the use of hash tables and linked lists. Here, each string would generate a hash value, which would then be used as the index for where to store the word. If there is the case that there is a clash of two words, then the resolution technique would involve separate chaining to each node in the table. In this organisation a linked list would be used, which would be organised like an array that the other search methods used. This was decided over other such techniques such as open addressing, as it would keep the number of hashing indexes low and be much easier to design and search.

The benefits of this are that it gives a best-case scenario runtime of O(1), however depending on how well the hash table is organised and how many words congregate in one linked list, it could give a worst-case scenario runtime of O(n).

1.4 Literature Review

For some cases, the developing spell checkers and their implementations can seem quite simple and straightforward. As outlined in the 2015 paper titled ‘Spell Checker’ [2], the obvious choice would be to use the Levenshtein distance algorithm to calculate the difference between an entered word and a word in a generated dictionary. The authors argue that when using this method, a spell checker can produce a list of suggestions by comparing a misspelled word to those in a dictionary that the program has access to. They also make the point to say that this algorithm provides easy identification of misspellings and can offer results very quickly, reducing the runtime of the application used. This study makes no mention of any other methods of creating such a spell checker, and therefore assumes Levenshtein to be the optimal choice for such an application.

However, an alternative to this theory can be found in the 2003 paper titled ‘A comparison of String Metrics for Matching Names and Records’ [3]. The difference here is that the paper focused on testing the searching of entities and structures over just words for a classic spell checker. They also focus on using a Java program to run their tests, which makes it similar to a C program given that it needs to be compiled to run. Regardless, they argue that the best method is the Levenshtein edit-distance metric, which they also modified together with the Jaro-Winkler Distance algorithm to give the best performance. This indicates that both standard Levenshtein and Jaro-Winkler are good models for searching a dataset accurately and are therefore the best choice in a spell-checking program.

1.5 Proof of Concept

To test the viability of a spell-checking system, a prototype was created. This was made to be as simple as possible, to gain a general understanding of how such a program would work. It would also assess if results could be retrieved in a reasonable time frame, thereby proving that a solution for the project was possible.

For this the C programming language was used as it is a compiler language. This means that it can run much faster than interpreter languages like Python. It was also used because one of the main applications would be developed using C and the lessons learned from this prototype could be exploited in the final product.

1.5.1 Design

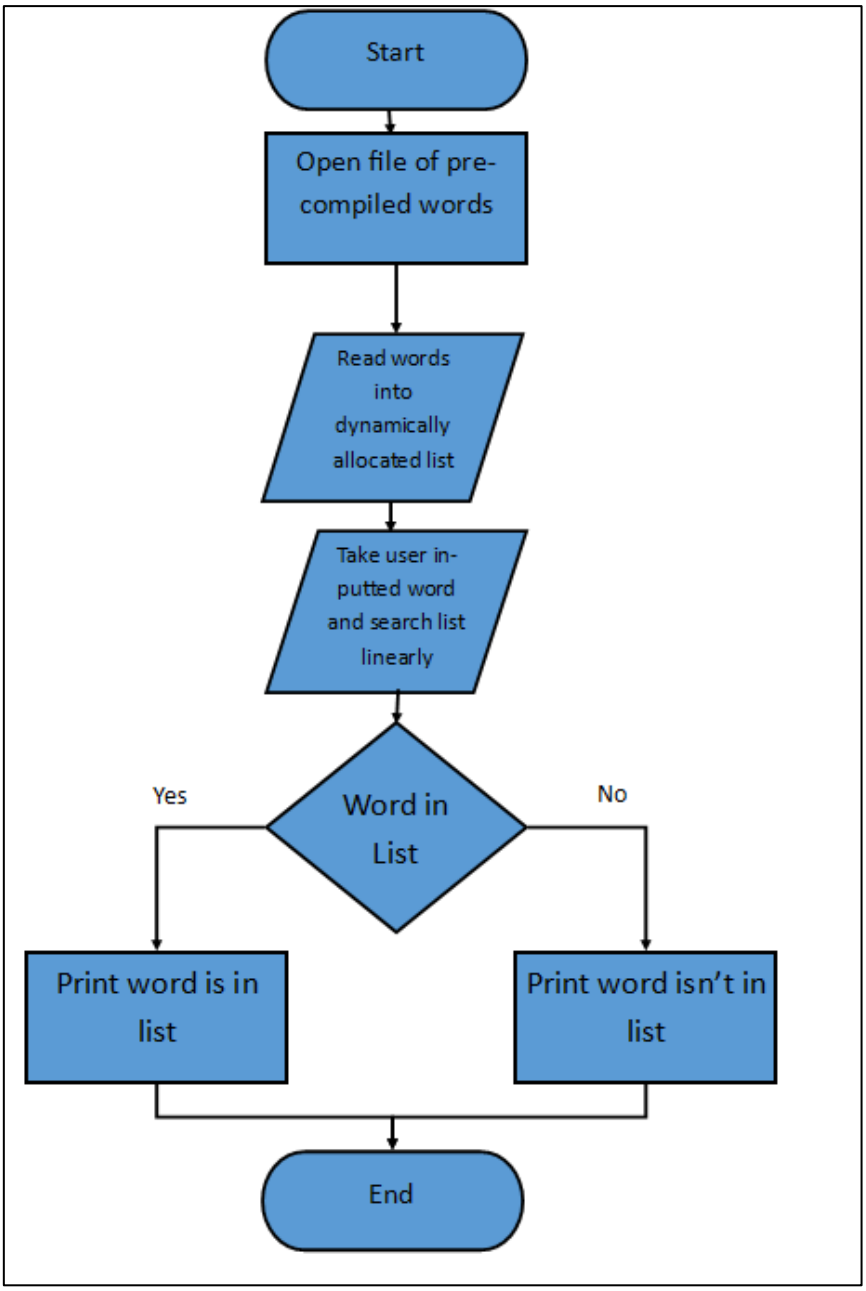

Figure 8. Flowchart for proof of concept

The proof of concept was very simple in nature, as the system would not perform any word list generation. Instead of having a corpus from which words could be extracted and then stored at runtime, this system instead assumed that words were already compiled and sorted in alphabetical order. To save time on the developing a word file, a precompiled dictionary was sourced from GitHub user dwyl [4]. Therefore, all that was needed of the code was to store the list in its entirety while running, then allow the user to enter a string, whereby the code would give a simple response as to whether the word exists in the list or not.

In this case, the prototype didn’t use any string metric algorithms. Rather, it was created to show that even a simple system can run and deliver results in a reasonable time to the user.

1.5.2 Results and Conclusions

The proof of concept ended up using a word list which contained 307,100 words. The file could have been optimised since many of the strings would hardly ever be used in modern English. Many were there just to give an exhaustive list of all possible word regardless of if they were used today.

The word list could have been optimised by using stemming. This also ties into why the file was so large, as it stored all permutations/conjugations of a word. Whereas if stemming was used, then these words would be reduced to their base form and therefore only one variation per word would be needed to be stored and checked.

Even with these imperfections and the large file size, the system could handle the data very easily. Though the runtime is dependent on the word entered, the system had a worst-case scenario runtime of 15.6 milliseconds. If even an inefficient system such as this can perform a check in such little time, then it would be possible to work on a more advanced and optimised system.

Chapter 2
Methods

2.1 Methodology

The original concept for the software was to develop a program, which would be split into multiple functions depending on what they were trying to accomplish. These functions would be developed one after another depending on how important they are to serving then next function. As an example, initially a lot of time was spent developing a simple command line user interface where users could type letters on their keyboards for navigating to perform a task such as compiling the words from a corpus. Without this step, it would be hard to test all the functions of the code at once.

As such, an Iterative development (Waterfall) methodology was used. This was done as it would allow some flexibility for the development of the final product. It was chosen because there would be only one person working on this product, which limited the amount of work that could be output in the development timeframe. As such it was considered wasteful to follow a more agile development process where multiple prototypes would be made and then discarded. Instead, the same software would be designed, implemented, tested, and then further adapted and optimised over time to meet all the requirements initially outlined.

The development of the systems would be backed up using a GitHub repository, which handled all the version control for the software in case the main development machine got corrupted in some way.

2.2 Corpus Extraction

The first sprint for developing the program revolved around the extraction of words from the corpus. This was not done to be tested for efficiency, but more as a means of generating a dictionary file for the actual spell checker to use, and assessing if the nature of its organization had any effect on the final efficiency of searching.

```
Corpus Extraction Pseudocode:

        //(A) Allow duplicates
        //(B) Don't allow duplicates
        //(C) Allow duplicates and sort in alphabetical order
        //(D) Don't allow duplicates and sort in alphabetical order
        //(Z) return to main menu
        print options for file generation to the user

        //start at corpus generation menu
        let Action := input character from user input

        if Action = A or B or C or D then
                DataReader(Action)
        else if Action = z then
                return to main menu
        else
                loop until valid answer is given

function DataReader()
        input: char Action
        output: file of words

        depending on the Action, the system will check for duplicates / allow sorting

        //get the users choice of corpora
        let CorporaChoice := input character from user input

        //generate the data file
        for subfolder in corpora do
                for file in subfolder do
                        read all valid words from the file
                        if duplicates are allowed then
                                check that any new words don't exist in the list before storing

        if sorting is allowed then
                sort the whole word array in alphabetical order

        open new file

        write all words in array to the new file

        close new file

        return
```

Figure 9. Pseudocode of Corpus Extraction process

It was decided that the user could generate 4 different types of word files to test the spell checker with. These included: allowing duplicates, not allowing duplicates, allowing duplicates and sorting in alphabetical order, and not allowing duplicates and sorting in alphabetical order.

The reason why the user is given the option to generate a word file which allows duplicates is to give an option for quick word file generation. This was created purely for testing purposes. Given that the code wouldn't need to check for duplicate strings, then the time to generate a new file would be shortened. Therefore, it would be better if you were wanting to quickly get a standard word file to perform tests with.

Then when it came to sorting the word file, this is done to check the performance for searching words for if they are arranged alphabetically. It is also done because one of the methods, the binary search, can only work if the dictionary is sorted.

The corpus chosen came from the British National Corpus (BNC) [5] and contains a series of folders labelled from A through to K. Each was full of xml files relating to various texts of different genres and content. It was decided that using this content would be much better than a list of text files as the BNC had taken the time to also define the type of each word, such as if it was a verb, adjective, etc. This would be useful in the long term as it would

prevent the need of developing a function to classify words. Therefore, if a system was to be implemented which could analyse the syntax of sentences and longer English texts, then the job would be a lot easier.

When selecting which words were valid or not, the corpus extractor would follow a strict policy. It didn't store single letter words, which is self-explanatory. It also only stored words in lower case, and disregarded hyphenated words and strings with apostrophes and other such characters. This of course would mean that the spell checker wouldn't be able to handle these types of words. However, this was done to simplify the storage of words, since the way the corpus worked made it hard to use words of this type.

2.3 Spell Checking

The second sprint was focused on the development of the actual spell-checking program, which included the coding of all the search methods and algorithms which would be tested against each other for efficiency. This part of the system was organized into a series of menu's which allows a user to specify everything including the word file, the search method, and the algorithm for suggestions.

```
Spell Checker Menu Pseudocode

        //check if the user has generated a word file already
        if file doesn't exist then
                return error

        else
                let FileNumber := number of file type the user wants to use

                let MethodNumber := number of method type the user wants to use

                let AlgorithmNumber := number of algorithm type the user wants to use

                let errorCheck = SpellChecker(FileNumber, MethodNumber, AlgorithmNumber)

                if errorCheck = an error code then
                        return errorCheck
                else
                        ask the user if they want to return to the main menu or not


function SpellChecker() is
        input: integer FileNumber, integer MethodNumber, integer AlgorithmNumber
        output: integer returnCode

        //generate the data structures depending on the users request
        if methodNumber refers to linear search or binary search then
                load file using FileNumber into an array

        else if methodNumber refers to hash table with linked lists then
                load file using FileNumber into a hash table with linked lists

        while user wants to enter a new word do

                let userString := word user wants to spell check

                if userString is valid then

                        start a timer

                        perform search for word depending on the method as indicated by the MethodNumber

                        if the word exists in the data structure then
                                print that it exists

                        else
                                print that it doesn't exist
                                if AlgorithmNumber != none then
                                        // create an array to store the distances between the user word and all other words
                                        let algorithmArray := [0 ... WordCount] be a 1-D array of integers of size wordcount

                                        for i := 0 to WordCount inclusive do
                                                using algorithm defined by AlgorithmNumber do
                                                        algorithmArray[i] = distance between userString and dataStructure[i]

                                        print out 10 suggestions which are closest to the userString

                        end timer and print time taken to find word
```

Figure 10. Pseudocode of Spell Checker process

Once the user has selected all the actual parameters for their search, they are then given a prompt to enter a desired word that they wish to test. This word must be valid, i.e. no numbers or non-letter characters. After being entered and validated, the system will load the words from the file type specified into a data type that the method chosen can handle. Then when everything is in place, the user's string is searched for. If it exists in the data structure, then no more processes are needed. However, if not, then the code will loop linearly through the data structure and use the algorithm called for by the user to calculate the distance between the user word and every other word in storage. Once all these values are compiled, the top ten closest words by distance are then displayed to the user. Finally, the time to compile all this data is displayed to the user.

This way, the user can get complete control of what they want to check, with the two main metrics of speed and suggestion similarity being shown immediately after completion. This allows for easy assessment of how effective the algorithm was in correcting the spelling mistake.

2.4 Automated Testing

The final sprint was dedicated to the development of a function to be able to see all the algorithms work together to make comparing results much easier and more accessible.

```
AutoTester Menu Pseudocode

        ask user to generate word file
        let FileNumber := the number associated with the file generated

        let errorCheck := AutoTester(FileNumber)
        if errorCheck = error code then
                return errorCheck


function AutoTester() is
        input: integer FileNumber
        output: integer returnCode

        //perpare the data for tests
        load file using FileNumber into an array
        load file using FileNumber into a hash table with linked lists

        while user wants to enter a new word do

                let userString := word user wants to spell check

                if userString is valid then
                        //loop though every algorithm in system
                        //expect when doing binary search on an unsorted file
                        if method = binary_search and generatedFileType = unsorted then
                                skip method
                        for method in system do
                                //for every method test every algorithm on it
                                for algorithm in system do
                                        start a timer

                                        perform search for word depending on the method as indicated by the MethodNumber

                                        if the word exists in the data structure then
                                                print that it exists

                                        else
                                                print that it doesn't exist
                                                if AlgorithmNumber != none then
                                                        // create an array to store the distances between the user word and all other words
                                                        let algorithmArray := [0 ... WordCount] be a 1-D array of integers of size wordcount

                                                        for i := 0 to WordCount inclusive do
                                                                using algorithm defined by AlgorithmNumber do
                                                                        algorithmArray[i] = distance between userString and dataStructure[i]

                                                        print out 10 suggestions which are closest to the userString

                                        end timer and print time taken to find word
```

Figure 11. Automatic testing function pseudocode

As such, an additional new interface would be needed for the program. Here the user generates a new word file just as you would using the corpus extraction function. Then once complete, the user gets a prompt for a word they wish to check exists or not. The same process used in the spell checker function gets employed, except all method and algorithms is tested at once. The results for each will then be displayed in a table, to easily describe how quickly each process ran as well as what suggestions, if any, they delivered.

However, as to be outlined later, it became apparent that to better show the capabilities of each search method and algorithm, a new application would need to be created. This built upon the automated testing process but could instead run the same tests in a loop while using list of known misspellings as a test set. This would then require a further application that would be able to read the results of these tests and then compile a series of graphs that could visualise the results clearly and in a way that was easier to understand.

Chapter 3
Implementation and Results

3.1 C Program

It was decided that to better gauge the speed of the algorithms at runtime, that two variations of the same program were drafted. One in C, and one in Python. With C being a compiled language and Python being an interpreted language, it would help to see if there was any major difference in speed and storage complexity which could then determine and optimal solution for a future product.

3.1.1 Implementation

The first spell checker program was developed in C. This was chosen because the C language is compiled all at once before running, which was planned to give the fastest speeds for running the algorithms. This would be very useful for the project, as the code base would have to be able to deal with word comparison from a minimum of 4,000 strings to 100,000 strings and beyond. All of which needed to be done in a time frame as short as possible, since an actual industry product would need to be able to do the same thing in real time.

However, the nature of C being a fast and optimised language also made it harder to write the code. C has limited options for storing data dynamically. The main issue was storing all the words at runtime, which had to be done with a fixed size array which was big enough to work for any of the given corpora.

The language also made developing the corpus extraction process more difficult, requiring time being spent on a large series of loops which would read the files character by character. Furthermore, C has a limited number of premade libraries to use, which meant all the string metric algorithms had to be self-developed and tested for correctness, which took even more time to complete. Despite the overall speed of the C program being good, it took a lot longer to understand and develop this final product.

Another issue found concerned finding the top ten suggested words for any given misspelled string. The way the system worked was that after the search method failed to find a given word in the dictionary, it would loop linearly through all the stored strings and calculate the distance between them and the user’s word. Once completed, to find the smallest values the program would loop through the entire list linearly again and then get the smallest value each time. This meant that to find all these suggestions would require an additional ten full

loops through the entire dataset. This would eventually be a problem as testing would be done on corpora with an increasingly large number of words.

3.1.2 Testing

Most initial testing was done with one of the corpora, specifically from the ‘D’ folder. This was due to the small number of xml files to parse, which would give a total of 4395 non-duplicate words to use. This number meant that the program could extract everything in less than a second, and therefore made it easy to move immediately on to the spell checking / algorithm analysis process, which is what was being assessed. For subsequent tests much larger corpora were used, which could usually take a couple minutes to parse even with the immense speed of the C language.

There was however a drawback to using this small subset of words, in that there were instances where you entered a word for an algorithm and the result was so instantaneous that the system reported a runtime of 0 milliseconds.

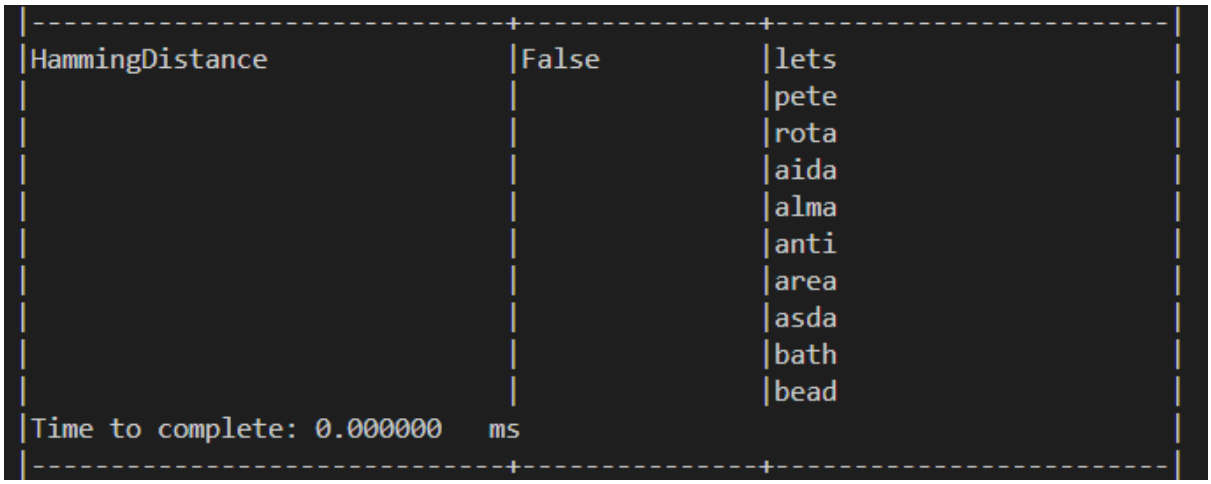

Figure 12. Extract of C program automated test showing runtime of 0 milliseconds.

This could have been considered an error if not for the fact that in other cases the other algorithms and methods were showing positive times. This indicates that the program was able to execute so quickly that the timer didn’t regard it as significant enough to produce a positive run-time.

As a result, later testing was done on the larger ‘H’ corpus, which contains a total of 136,000 non duplicate words. This was chosen as it was the largest corpus to use, which would help in assessing the capabilities of the system better than with the ‘D’ corpus. This would hopefully reduce the issue of zero runtimes and more clearly define the benefits and drawbacks of each search method and algorithm to handle such a large dataset. Furthermore, it would increase the chance that the correct spelling for a word did exist in the list, which would be useful in the automated testing stage of the program to ensure the results were as fair as possible.

To test the capabilities of each algorithm properly, a dataset of misspelled words was used from Roger Mitton [6]. His collection contained a multitude of files containing misspellings and their correct equivalents, which could be fed into the system one after the other. After all these words were checked, then each search method and algorithm could be analysed for how fast they ran and if the correct spelling appeared in one of the top 10 suggested words. To further examine the effects of different word organisations, two tests were conducted. One with a sorted dataset and one without.

In the end however, only one file of his was used which contained over 508 misspelled words. There were other larger test sets of 4,000 and even 40,000 words, however these were soon rejected due to the performance of the Python program to keep the test results fair. With this small dataset, the C program could completely compile the results in around 20 minutes. This extended time can be attributed to the fact that for every one of the misspelled words, the system tested each 3 search methods, where for each method 8 algorithmic tests were performed. Combine that with the implementation issue of having a minimum of 10 full loops though the whole set in order to find the suggested word, and you can understand why it takes so long to complete testing.

3.1.3 Results

Once all the tests had been completed, a Python program was developed to compile and display the results in a series of graphs using the library matplotlib [7]. These were divided based on three criteria for assessment, which were runtime, suggestion number, and miss counts.

Both the sorted and unsorted dataset tests produced similar runtimes, so it was easy to compare all three methods together. From looking at the various times in figure 13, a general trend appears between each of the algorithms. First, binary searching provides the fastest runtimes as given by the fact that the average for each algorithm is lower compared to the others.

Secondly, algorithms such as Hamming Distance and Sørensen–Dice are on average very fast given their low mean time, compared to the much slower algorithm of Damerau-Levenshtein. This in turn confirms the time complexities of each of the algorithms since the former two algorithms are also the fastest compared to the others.

However, the case of Jaro and Jaro-Winkler contradicts their as it has a complexity of O(m * n) yet keeps its average runtime lower than Levenshtein Distance which should run in the same quadratic time frame. This could be due to the implementation of the Jaro algorithm, which can break from the calculation loop if a string doesn’t meet certain criteria. By contrast, Levenshtein doesn’t have such a feature and must complete the full loop each time.

It is also clear that the more complex the algorithm is, the greater its variance is at runtime. This is since the Inter-Quartile Range (IQR) for both Damerau and Optimal String Alignment is a lot larger than its counterparts. This is most likely because these two algorithms must perform four separate operations for each letter, so the number of checks can vary depending on a word's length.

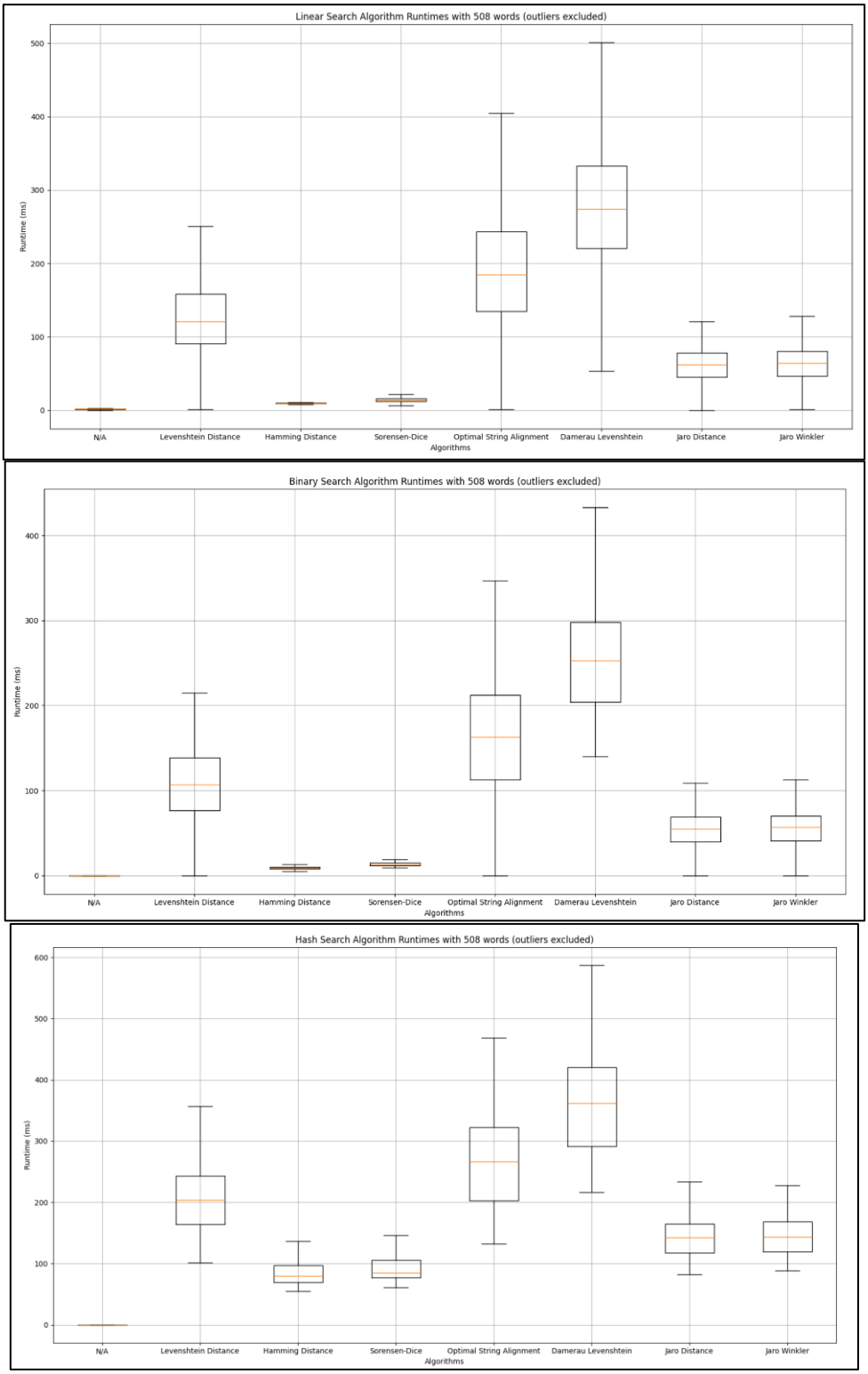

Figure 13. Runtimes for C program with each search method and algorithm in C

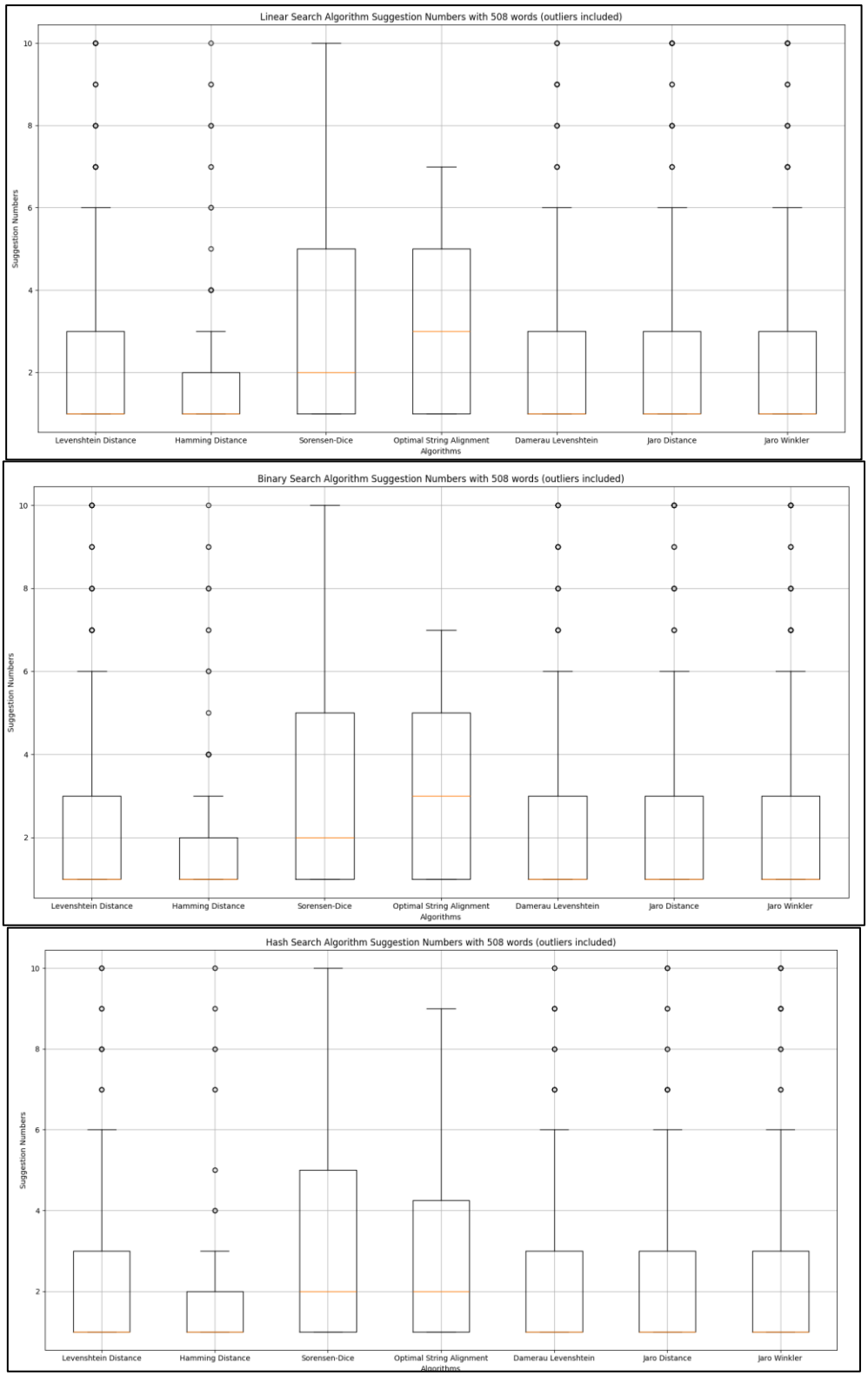

Figure 14. Comparison of suggestion numbers with each search method and algorithm in C.

When looking at figure 14, we assess the average suggestion number of each algorithm and search method. This number refers to where in the top ten suggestions the correct spelling for a misspelled word was. So, if the average is close to one, that means that for any given

misspelled word, its correct spelling was usually the top suggested word. There is no variation between the three search methods as to their suggestion scores since they have no bearing on the results.

From the figure we can see that the average suggestions where best for all but Sørensen–Dice and Optimal String Alignment. This may be because these algorithms are not considered metric distances are therefore less accurate in their results than the ones that do obey. It could also be the result of random error in the implementation of the algorithm.

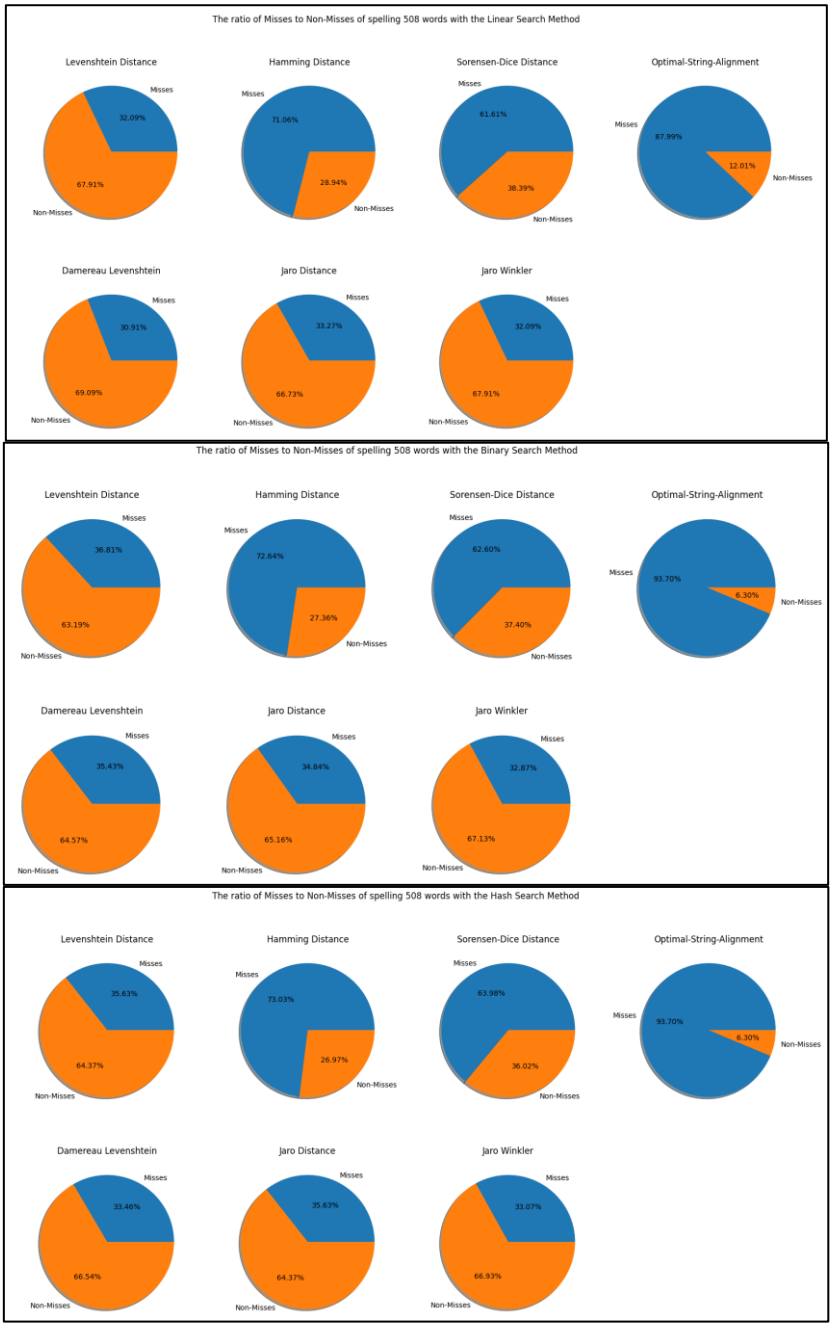

Figure 15. Comparison of the ratios of misses to non misses for each search method and algorithm in C.

Then finally, from figure 15 we can see the miss counts for each of the search methods and algorithms. This measures the ratio of how many times the correct spelling for a word didn't appear in the top ten search results (miss) compared to how many times they did (non-miss). Again, the three search methods have very little effect on the outcome, so results are dependent on each of the algorithms.

Here another trend appears, with algorithms such as Levenshtein, Damerau, Jaro and Jaro-Winkler being the most likely to find the correct spellings. This compares to the other three, which are each more likely to miss a correct spelling. With Hamming Distance this would be because it is restricted to find words of the same length, and as such won't find spellings where extra letters are needed. However, with Sørensen–Dice and OSA, this most likely ties into the same fact as stated with the suggestion number. As they are not true metrics, they therefore cannot find results as reliably.

Therefore, to give an overall assessment of the best performing algorithm, the Jaro and Jaro-Winkler algorithms seem to be the solid choice. This is shown by the fact that they run in some of the shortest times out of any of the algorithms, giving it an advantage over the others. Its usefulness if further compounded by the reliable average suggestion numbers and relatively low miss-counts, especially when looking at faster algorithms like Hamming and Sørensen–Dice.

3.2 Python Program

3.2.1 Implementation

The python version was worked on after the C program was nearly completed. Therefore, development lessons and ideas could be copied and adapted, which saved a large amount of time creating the application.

Coding was a lot easier with Python, given that it has access to many libraries and data structures that C lacks.

One of the most helpful libraries was BeautifulSoup [8], which could do all the word extraction from the xml files in a few function calls. Python's dynamically sized array and dictionary data structures were also invaluable in holding the large quantity of words at runtime. All these benefits cut down the amount of time spent coding compared to C, which helped the final product be delivered in such a short timeframe.

Another useful discovery was the library strsimpy [9], which had several of the string metric algorithms already created and ready to use which included the Levenshtein, OSA, Damerau-Levenshtein, Sorensen Dice and Jaro Winkler algorithms. These were eventually used over self-generated versions since they were more easily accessible and were

guaranteed to work with most scenarios and comparisons, as well be more likely to produce optimal solutions.

3.2.2 Testing

As with the C program, the main corpus that was used for testing during development was the ‘D’ corpus, given its small size and word count which could be parsed in around 2 and a half seconds. This was done so that testing could be done swiftly and without having to wait for larger corpora to be compiled. With this version, there was no issue with the timer and no zero results.

As with the C Program, later testing was done on the larger ‘H’ corpus. This was chosen to keep the testing of both systems fair and to have the benefit of having many spellings to call upon to prevent the risk of a valid word not appearing in the list. However, despite the help of the external libraries of BeautifulSoup [8] and the easy development of Python, it would run a lot slower. Compared to the couple of minutes of the C code, this version took around an hour to retrieve all the words. To speed up the testing process, it was decided that it would be better to use the dictionaries generated by the C program, then copy them into the Python folder to save time.

To test the capabilities of each algorithm properly, and to keep the results of testing fair, the same dataset of misspelled words was used again. Both sorted and unsorted word files were also analysed to test that as a factor as well. A similar system was adopted to test everything. For every misspelled word, each method was run with all algorithms giving a total of twenty-four tests per word. The same issue of finding the top ten suggestions also proliferated here, because the spell-checker had to find the minimum for each test ten times, which meant there was a large amount of processing needed to return even a single result.

This combined with the slow runtime of Python meant that to even test the 508 words of the smallest misspelled dataset took hours to complete. This was why the larger files were unable to be used. While the C program could have returned results in a reasonable time, the Python would be unable to do the same.

3.2.3 Results

As with the C program, the same three metrics of runtime, suggestion number and miscounts were analysed. Here, whether the dataset was sorted or not didn’t affect the results and so the generated graphs show the general cases for each metric and search method.

Firstly, there is a stark difference between the runtimes of the C program and this version, as indicated by the greater averages for each algorithm. This is especially apparent for the quadratic algorithms of Damerau and Optimal String Alignment (OSA), which have a mean

time over ten times that of its C counterpart. It is therefore clear that Python doesn’t run these actions swiftly.

Despite this, the general trend of the run times for each algorithm remains the same as before, with the fastest remaining Sørensen–Dice and Hamming distance for their linear time complexity. Whilst the larger algorithms of Damerau and OSA take longer on average, due to them having to loop through a full matrix to compute their distances.

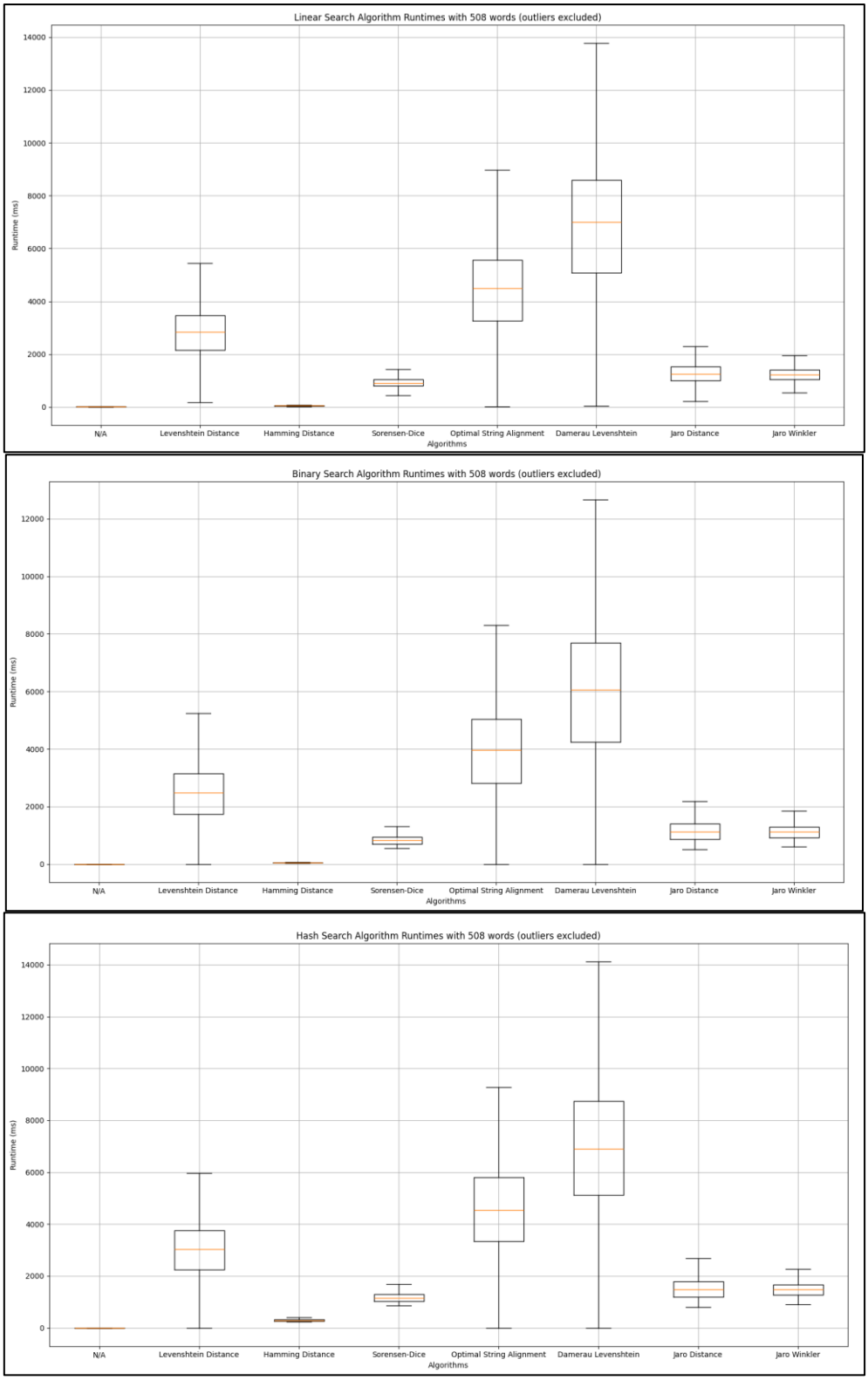

Figure 16. Boxplots of the run times for each search method and algorithm in Python

Secondly, the search methods once again don't have a bearing on the results of the suggestion numbers. So, when looking across the algorithms their numbers indicate that apart from Sørensen–Dice they all average at one, so they are all very likely to give the correct spelling to a typo as their first suggestion. The only major difference between this

and the C code's performance regards OSA. Unlike the self-developed version, the strsimpy python function seems to be more correct as shown by the fact that its average and IQR are the same as many of the others. This could be indicative of the fact that the version of OSA is more reliable than the self-made C alternative and is therefore able to produce better results.

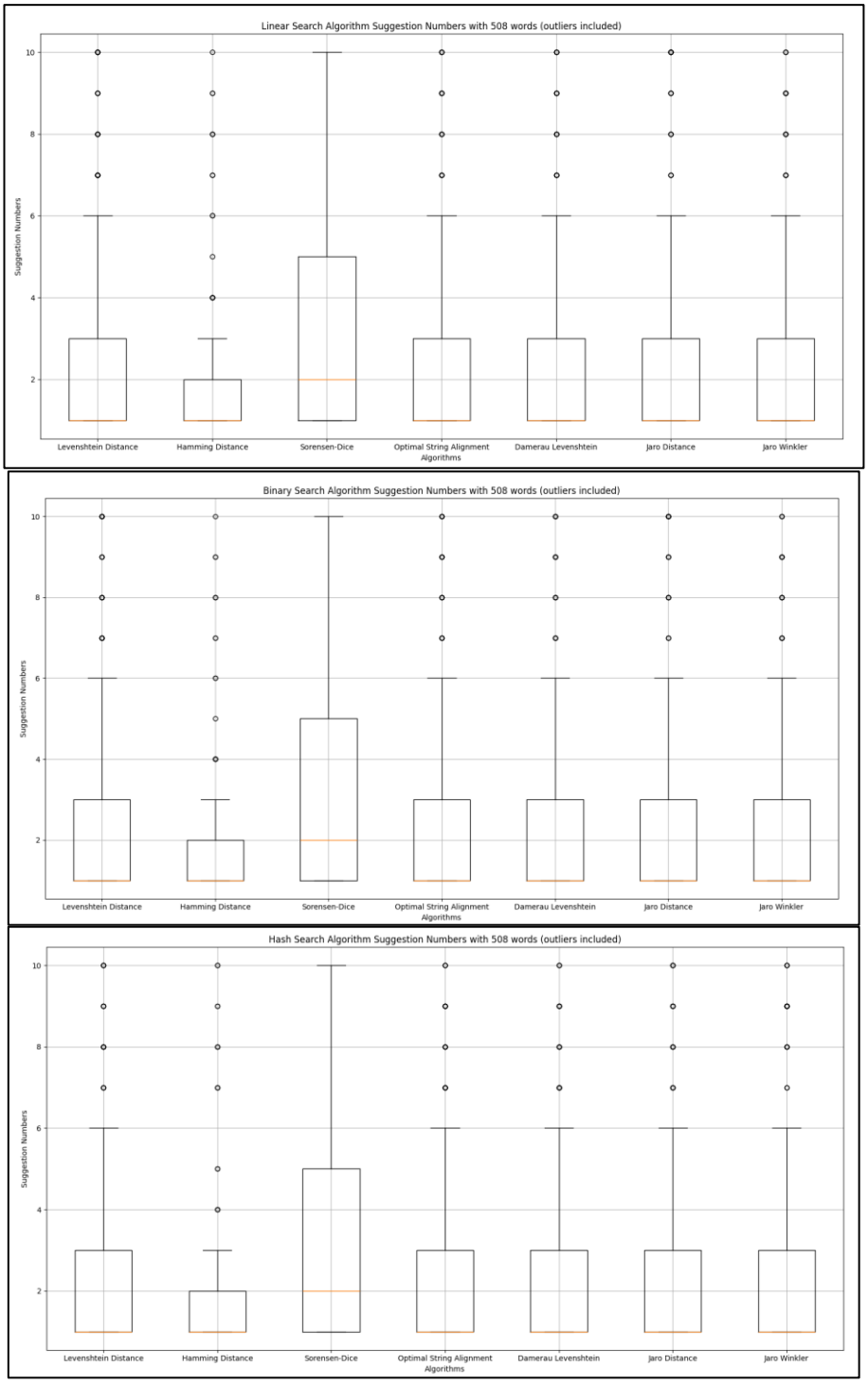

Figure 17. Boxplots of the suggestion numbers for each search method and algorithm in Python

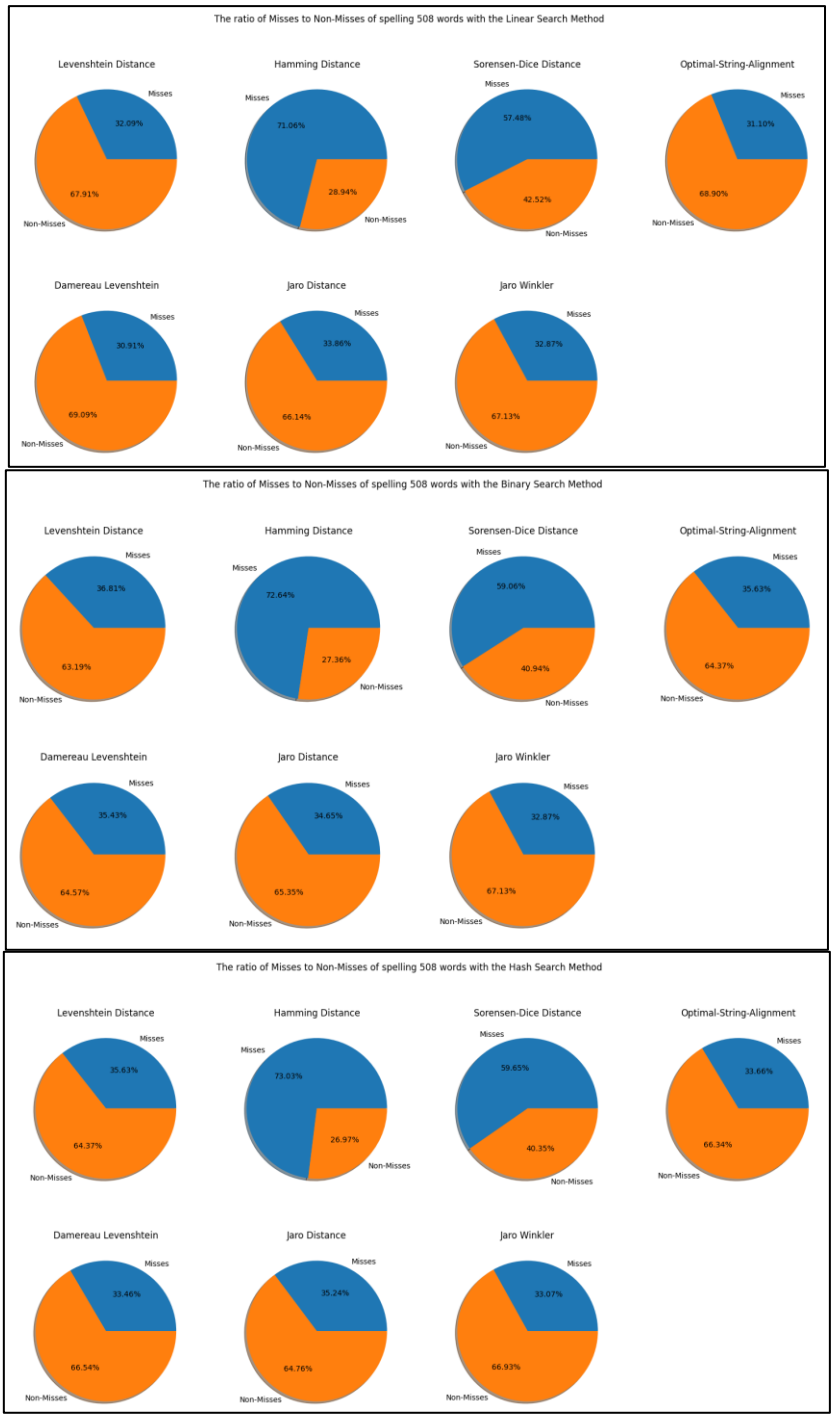

Figure 18. Pie charts showing the ratios of misses to non-misses for each search method and algorithm in Python.

Finally, we look at the miss and non-miss counts from figure 18. Here, the search methods do seem to affect the percentage of misses to non-misses in a small way, with the ratios of missed being lowest with linear searching and higher with binary searching. This could be indicative of some small errors in implementation or perhaps an element of randomness regarding how the data results are collected.

Then, comparing each algorithm we can assess a major difference with the OSA algorithm again. The C version of it had the highest miss rate of all the algorithms, whereas here it has a ratio like those of Levenshtein and the other algorithms of similar complexity. This further reinforces the idea that this Python implementation of OSA is better and gives more accurate readings.

Despite that, the general trend of ratios for the other algorithms are like that of the C program. Now, the worst performing algorithm is Hamming Distance. This is because of its limitation of requiring strings of equal length to work, discarding all others in the process. However, the difference between the other algorithms is very small with each having a very similar number of misses to non-misses. This makes it harder to dedicate a better choice. It can be said that Jaro-Winkler is the most consistent algorithm across the three search methods, with a high non-miss score that indicates that it is one of the better algorithms for finding the correct spelling of the word you were trying to search for.

Chapter 4
Discussion

4.1 Possible Improvements

The system would have benefited from word stemming. That would involve an automatic check on each new word parsed by the corpus extractor, which would reduce the inflected string down to its root. Without needing to store all this extra data, the number of words could have been cut drastically, which would have greatly improved the performance of the systems.

Another step could have been to test various adjustments to the search methods and algorithms. The major issue with the final product was that searching involved multiple full loops of the word list. To make this more efficient, certain steps could have been taken such as to limit the loop to searching words that start with the same letter. Small improvements such as this would reduce time for the spell-checking process and make the issue of having to wait so long for results more acceptable.

4.2 Conclusions

The main aim of this project was to assess the best possible methods and algorithms needed to optimise the performance of a spell-checking software.

From the numerous tests conducted, the C program was the better choice when looking at the performance of the algorithms. With one of the metrics being runtime, without a doubt the compiled nature of C gave it an advantage that Python didn’t have. This makes it the better choice for a spell-checking application. It is recommended that for any similar systems generated in the future that a compiler language such as C or Java is used.

Then, when looking specifically into the runtimes of each search method, the logarithmic time complexity of the binary search was the better choice for speed. It was therefore apparent that a sorted dataset was needed for optimal search speeds and for the binary method to function.

From comparing the results of each algorithm, many trends appeared to indicate how in certain scenarios some methods perform better than others. However, it was clear across both the Python and C programs that the Jaro-Winkler distance algorithm was the better option for typo correction. It could perform checks in a reasonable time frame and provide suggestions that were incredibly accurate.

It is therefore recommended that any future spell-checker applications should follow the above suggestions. To get the best results for a consumer product, a program should run on precompiled software, using a sorted dictionary with the binary search method to find words efficiently. Then to find suggestions for typos the Jaro-Winkler distance algorithm is preferable for more accurate corrections.

4.3 Ideas for future work

The next stage of development would involve the creation of a natural language processor which could handle the checking of multiple words in a sentence in context. This could be extrapolated into making a system which would take series of texts, which them could be looked at and used to return any errors.

Furthermore, the list of tested search methods and algorithms was far from exhaustive. There are certainly other options which could have been assessed to see if they give more accurate results in a better time frame. Once such method that was considered but never tested was the use of parallel searching. Here a system could make use of a multiple core processor to compare words simultaneously to reduce the runtime of the spell-checker. There also exist further string metric algorithms such as the Jaccard index, Cosine Similarity, etc. In a future development these other ideas could be included to help assess if any other optimal solutions to the problem exist.

List of References

[1] Oxford University Press. 2023. Oxford English Dictionary. [Online]. [Accessed 30/04/24]. Available from: https://www.oed.com/?tl=true

[2] Bhaire, VV, et al. 2015. Spell Checker. International Journal of Scientific and Research Publications.[Online] [Accessed 18/03/24]. Available from: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=939835023700f66cc7fbb861a76af93824b3b082

[3] Cohen, WW, et al. 2003. A comparison of String Metrics for Matching Names and Records. [Online]. [Accessed 30/04/24]. Available from: https://www.cs.utexas.edu/~ai-lab/pubs/kdd03.pdf

[4] dwyl. 2023. English words files. [Online]. [Accessed 02/01/24]. Available from: https://github.com/dwyl/english-words

[5] BNC Consortium. 2007. British National Corpus, xml edition, Literary and Linguistic Data service. [Online]. [Accessed 29/01/24]. Available from: https://llds.ling-phil.ox.ac.uk/llds/xmlui/handle/20.500.14106/2554

[6] Mitton, R. 2024. List of misspelled files. [Online]. [Accessed 30/04/24]. Available from: https://www.dcs.bbk.ac.uk/~roger/corpora.html

[7] pypi. 2023. Matplotlib Python visualisation library. [Online]. [Accessed 26/04/24]. Available from: https://matplotlib.org/

[8] pypi. 2024. BeautifulSoup Python library. [Online]. [Accessed 30/04/24]. Available from: https://pypi.org/project/beautifulsoup4/

[9] strsimpy. 2024. Strsimpy Python library. [Online]. [Accessed 26/04/24]. Available from: https://pypi.org/project/strsimpy/

Appendix A
Self-appraisal

A.1 Critical self-evaluation

I believe this project has successfully completed the aims and objectives it sought to achieve. This includes: the introduction, design, implementation of the two applications and then their subsequent evaluations. As such I have shown that the Jaro-Winkler algorithm is the best spell-checking algorithm in both speed and accuracy. I have also described the benefits of using the binary search on a sorted dictionary to find a given users word in an optimal time frame.

I acknowledge that I am disappointed the data set I had to use was smaller than I would have preferred. I was hoping that both applications could have used Roger Mitton's 40,000 misspelled word file. With such a large dataset, it would have made the results more reliable by reducing random error. It would have also shown how capable the system was in being able to handle so many spellings in a given time.

However, the time that it would have taken to compile these results in the C program, let alone the Python program, would have been too long without the assistance of a high-performance machine. This would have then defeated a point of the project, which was to show that there exist efficient algorithms for a spell-checking application, which would be expected to run on an any given machine.

I had planned to test more search methods. One such example that was discussed with my supervisor included searching using parallel computation. This could have been used as an alternative to the linear search method and would have sped up search times for finding suggestions.

I had also considered developing another search method using the tree data structure. From this I could have organised the words from a corpus into a tree, and then used a Breadth-First or Depth-First search method to assess if that had any bearing on improving the runtime of the result.

In the end I was unable to test these theories due to the time constraints of the project. However, it is also worth noting that it took a long time to run all the search methods and algorithms outlined already. So, adding even more to test would have been a challenge.

A.2 Personal reflection and lessons learned

All the code I wrote in C was new, which took a long time to test. This included everything from the algorithms and search methods to the user interface. The prototype helped to conceptualise the problem and understand what needed to be done, however its limited scope meant it contributed little to the final C code.

As for the Python program, it was quicker to develop as I was able to copy the interface and general function structures over quickly. I initially attempted to develop the algorithms myself the same as I did in C. However I soon found the strsimpy library, which had developed the algorithms already and could easily be utilised. I then decided to use these instead of my own versions to save time and guarantee that my results were correct. This turned out to be the best choice, as it allowed more time to develop the automated testers for each application.

Despite the difficulties experienced through the development cycle, I was continually trying to add additional search methods and algorithms. I had even set myself a stretch goal of making a full spell-checking program that could have worked in sentences and paragraphs. This was why I chose to use the files from the British National Corpus. They had already assessed the type of each word in their xml files. This meant that I could have made a natural language processor a lot more easily.

However, in the end I was encouraged to develop a system in Python because of its data structures and vast array of libraries. Given the knowledge now that such an application is incredibly slow, I should have used the time I spent making the Python code to developing a natural language processor in C. This is given that it would have produced the fastest results.

Regarding the management of the project, the use of GitHub was very useful in providing a backup of my work. It was also useful to see the lifecycle of the program and use it as a metric to examine if I was on track. However, I would argue that I didn’t use it to its fullest extent, as for instance setting scrum goals and automated tests to the main branch. In future projects, especially those in groups, I would look to use these features to streamline the development process.

A.3 Legal, social, ethical, and professional issues

A.3.1 Legal issues

For legal issues, the program did make use of multiple external libraries for the python program and texts to test the code. These have been referenced in both the project writeup

and the readme file of the GitHub repository, so as to credit the original owners for their work.

A.3.2 Social issues

There are no social issues associated with this project. This is because the efficiency of a spell-checker has no direct impact on society.

A.3.3 Ethical issues

All datasets and libraries were publicly available and were referenced appropriately. Furthermore, with the nature of this project being about algorithms and search methods for a spell checker, this means there were no ethical issues regarding the project.

A.3.4 Professional issues

This project was only developed by myself, under the watch of my supervisor Ammar Alsalka and assessor Brandon Bennett. Therefore, there are no professional issues associated with my project.

Appendix B
External Materials

All datasets used by the applications were downloaded off the internet from publicly available sources as listed below:

- The list of words for the prototype was retrieved from GitHub user dwyl at: https://github.com/dwyl/english-words
- The collection of texts that the corpus parser used were retrieved from the British National Corpus at: https://llds.ling-phil.ox.ac.uk/llds/xmlui/handle/20.500.14106/2554
- The files of misspelled words were acquired from a Roger Mitton from his website at: https://www.dcs.bbk.ac.uk/~roger/corpora.html

Below is the link to the GitHub repository which contains all the files and programs made during the development lifecycle:

https://github.com/uol-feps-soc-comp3931-2324-classroom/final-year-project-MatthewAlainCamus